Java Fundamentals

By Jawad Jammoul

Content:

Chapter 01 : Elementary Programming:

01*About Java:

What is Java?

Java is a high-level , object oriented programming language used to build cross-platform applications.

Why Using Java?

Java is a popular because it's platform-independent , secure , robust , and has a large community and rich libraries.

02*Definitions:

Class:

A class is a template or a set of instructions for creating something , it specifies what data it will handle and what operations can be performed on the data.

Main Method:

Is the entry points of any stand alone applications . It's where the program starts its execution .The main method is always defined inside the class.

```
public class className {
  public static void main(String[] args) {
    System.out.print("Text");
     } //end main method
  } //end class
```

Output
Text

Attributes:

In the main method any method in Java , attributes (or variables) are used to store data and hold values that can be used and manipulated within the method.

```
dataType variableName ;
variableName = value;
//is Equivalent to
dataType variableName =  value;
```

```
public class className {
  public static void main(String[] args) {
    dataType variableName ;
    variableName = value ;
    System.out.print(variableName);
     }
  }
```

Output
Value

03*Data Types and Variables:

DataTypes:

Primitive : int , double , long , float , boolean , char …
Non-Primitive : String , Array , Structure , Classes …

Variables:

Declaring:
int x ; // declare x to be an integer variable
double y ; //declare y to be a double variable

Assignment Statements:

x = 1 ; //Assign 1 to x
y = 5.3 ; //Assign 5.3 to y
You can store an int in a double , but not a double in an int without casting.

Example:

double x =1 ; is Ok but int y = 3.2 ; is an error
You can add int and double into a double
You can't store the result of int+double in an int without casting

```
double x = 5;
int y = 3.2 ;
int a = 4;
double b = 2.5 ;
double c = a+b ;
int d = a + b;
int e = (int)(a+b) ; //ok
```

Caution!

```
int x = 12;
System.out.println("x") ; // print x (text)
System.out.println(x); //print 12 (value)
```

Concatenation operator(+):

```
int a,b,c ;
a= 7;
b= 3;
c =a+b;
System.out.println("Hey , the value of c is " + c);
```

Output
10

Final Modifier:

The final Modifier is used to define constants or to prevent modification , when a variable is declared as final , it's value cannot be changed once it's it has been initialized

```
final double a = 5.7;
double b = 1.3;
a= 1.3;
double c = a+ b;
System.out.print(c) ;
```

Output
Error

04*print Vs println:

print() outputs text without adding a new line at the end , while println() output text followed by a new line , moving the cursor to the next line.

```
System.out.print("Are you ");
System.out.println("Ready!");
System.out.print("for some ");
System.out.println();
System.out.println("Good Luck");
```

Output
Are you Ready! For some practice Good Luck

05*Character(Char) & String:

Character:

.The character data type (char) is used to represent a single character

.A character literal is enclosed is single quotation marks:

Char variableName = 'value' ;

Example : char x = 'A';

ASCII Table	
Dec : Decimal value Char : Character	
Dec	Char
64	@
65	A
66	B
67	C
…	…
97	a
98	b

Caution:
'A' is a character but "A" is a String

Char casting:
char c = (char)97 ; -> a
int i = (int)'A' ; -> 65

Char with Scanner:
char c = scannerName.next().charAt(0);

String:
A String is a type of data that represents a sequence of characters , like words of sequence of characters , like word or sentences .It is used to manipulate text.
String variableName = "Text" ; //String name = "Jawad" ;

String With Scanner:
If it is word -> scannerName.next() ; //Jawad
If it is a sentence -> scannerName.nextLine() ; //Jawad Jammoul

\n and \t :
\n : is an escape sequence used to represent a new line character , which moves the cursor to the beginning of the next line.
\t : is an escape used to represent a horizontal tab character , which inserts a tab space in the output

Exercise 01
System.out.print("Java is"); System.out.print("/n fun");

Output
Java is fun

Exercise 02
System.out.print("Java is /t fun");

Output
Java is fun

06*Errors:

Runtime errors:

A runtime error is an error that occurs while the program is executing , causing it to terminate unexpectedly due to illegal operations or invalid states . The division by zero is an illegal operation System.out.print(1/0) ;

Logical errors:

Example 1:	
System.out.print(3*4-3) ; Output : 9	System.out.print(3*(4-3)); Output : 3
System.out.print(12/4+2); Output : 5	System.out.print(12/(4+2)); Output : 2

Syntax errors:

A syntax error is a mistake in the code's structure that prevents the program from compiling , typically due to incorrect use of language rules or punctuation

Incorrect : system.out.print(Welcome to Java")

Correct : System.out.print("Welcome to Java");

07*Scanner & Comments:

The Scanner class is used to read input from various sources like keyboard input , files , or streams , providing methods to parse and process the input data

Example

```
import java.util.Scanner;
public class Name {
  public static void main(String[] args) {
    Scanner scannerName = new Scanner(System.in);
    System.out.println("Enter an integer:");
    int a = scannerName.nextInt();
    System.out.println("Enter a reel number:");
    double b = scannerName.nextDouble();
    }
}
```

Comments:
Comments are used to annotate code , making it easier to understand for others or for yourself in the future.

Types:
[Single-line comments] : Start with // and continues to the end of the line
[Multi-line comments] : Start with /* and ends with */ It can span multiple lines

Example

//System.out.println(“Good Morning”);
System.out.println(“Welcome to Java”);
/* System.out.println(“Let us start”);
System.out.println(“Have a nice day”) ; */

Output
Welcome to Java

08*Numeric Operators:

Name	Meaning	Example	Result
+	Addition	34+1	35
-	Subtraction	34.0-0.1	33.9
*	Multiplication	300*30	9000
/	Division	1.0/2	0.5
%	Remainder	20%3	2

7485%10	5
7485%100	85
7485%1000	485

7485/10	748
7485/100	74
7485/1000	7

{ }	curly braces
[ ]	square brackets
‘ ’	single quotation
“ ”	double quotation
,	comma
;	semi colon
.	Dot
:	colon

Increment	Decrement
x++ ; or ++x;	x--; or --x;
x=x+1;	x=x-1;
x+=1;	x-=1;

Arithmetic Expressions
(3+4x)/5 - 10(y-5)(a+b+c)/x + 9(4/x + 9+x)/y Is translated to (3+4*x)/5 - 10*(y-5)*(a+b+c)/x + 9*(4/x + 9+x)/y

5/2 yields an integer 2
5/2.0 or 5.0/2 yields a double value 2.5
5%2 yields 1
5/2 yields 2

09*Math Class:

a power b	Math.pow(a,b) ;
radical a	Math.sqrt(a);
Random nb between 0 and <1	Math.random();
PI = 3.141592653589793	Math.PI;

Math.Random() Arithmetic Expression
For example if you want a random nb between 3 and 10 :

```
int min = 3 ; int max = 10;
double x = min + (int)(Math.random() * (max-min+1));
```

Chapter 02 : Selections

01*Boolean:

A boolean in Java is commonly used for conditional statements(selections) and control flow to determine the execution path based on logical conditions , it is a primitive data type that represents one of two possible values: true or false
Example: boolean x = true; boolean y = false;
boolean variableName = value;

02*Comparison operators & logical operators:

==	equal to
!=	not equal to
>	greater than
<	less than
>=	greater than or equal to
<=	less than or equal to

&&	And
||	Or
!	Not
^	Xor

03*If-else:

One way if statements:
A one way if statement executes a block of code if a specified condition evaluates to true

```
if(condition) {
  //code
}
```

If an if statement executes a block of code only if a specified condition evaluates to true

```
if(condition)
  statement;
```

same

```
if(condition) {
  statement;
}
```

if(true) : the code inside will always execute
if(false) : the code inside will never execute

Two-way if-else statements:

A two-way if-else statement provides a way to execute different code paths based on whether a condition evaluates to true of false

```
if(condition) {
//code1 (if true)
}
else {
//code2 (if false)
}
```

Nested if-else:

A nested if-else statement is an if-else structure placed inside another if or else block to allow for multiple layers of conditional logic

```
if(condition1) {
  //code1
  if(condition1.1) {
    //code1.1
  }
  else {
    //code1.2
    }
  }
else {
  //code2
}
```

Alternating if-else:

An alternating if-else statement sequentially checks conditions and executes the corresponding block of the first true condition

```
if(condition1) {
  //code1
}
else if(condition2) {
  //code2
}
else if(condition3) {
  //code3
}
else {
  //code4
}
```

04*Common Errors:

Common errors in if-else statements include missing braces , incorrect condition expressions , and improper nesting

if(condition);
//code;
	if(condition) {
	}
	//code;

Equivalent to	

05*Switch:

A switch statement is used to execute one block of code among many options based on the value of an expression

```
switch(expression) {
  case value1:
     //code1
     break;
  case value2:
     //code2
     break;
  case value3:
     //code3
     break;
  default :
     //code4;
     break;
}
```

Chapter 03 : Loops

01*Why using loop?

Loops are used to repeatedly execute a block of code as long as a specified condition is true, making it efficient to handle repetitive tasks and iterate through collections or arrays

Example :

Write Hello World 100 Times
System.out.println(“Hello World!”); System.out.println(“Hello World!”); System.out.println(“Hello World!”); System.out.println(“Hello World!”); …. …. System.out.println(“Hello World!”); System.out.println(“Hello World!”);

02*Loops:

while loop:

A while loop repeadlty executes a block of code as long as its specified condition remains true, allowing remains true , allowing for dynamic and flexible iterative based on changing conditions

```
while(condition) {
  //code to be executed
}
```

Same Example:

```
int i=0;
while(I<100) {
   System.out.printlnln(“Hello World!”);
   i++; //increment
}
```

for loops:

A for loop provides a concise way to to iterative over a range of values or a collection by initializing a counter , specifying a termination condition , and updating the counter on each iteration.

```
for(initializion ; condition ; update) {
   //code to be executed
}
```

Same Example:

```
for(int I =0 ; I<100 ; I++) {
   System.out.print("Hello World");
}
```

for(initialization;condition;update) { //one statement; }	for(initialization;condition;update) //one statement;
Equivalent to	

do-while loops:

A do-while loop executes a block of code once before checking its condition to execute the code as long as the condition remains true , ensuring that the code inside the loop runs at least once

```
do {
   //code to be executed
} while(condition);
```

Same Example:

```
int i=0;
do {
   System.out.println("Hello World");
   i++;
} while(i<100);
```

03*Difference between i++ and ++i :

++i increments the value of i before it is used in an expression, while i++ increments the value of i after it is used

Example:

```
int i=5,j=5;
System.out.println("i :" + ++i);
System.out.println("j:" + j++);
System.out.println("Final i :" + i);
System.out.prntln("Final j: " +j);
```

Output
i: 6 j: 5 Final i : 6 Final j : 6

04*Keywords break/continue:

break: The break Keyword is used to exit a loop prematurely , terminating the loop's execution and continuing with the code the follows it

boolean keep = true;

```
int i =0;
while(keep) {
if(i==3)
break;
System.out.println(i);
i++;
}
```

Output
0
1
2

continue: Skips the rest of the code inside the loop and jumps to the next iteration
It does not stop the loop , it just skips the current iteration

```
for(int i=1 ; i<=5 ; i++) {
  if(i==3) {
    continue; //skip when =3
  }
  System.out.println(i);
}
```

Output:
1
2
4
5

05*Nested Loops:

Nested loops are loops placed inside other loops , allowing for multiple levels of iteration over different ranges or conditions

Example:

```
for(int I=0;I<3;I++) {
    for(int j=0;j<3;j++) {
       System.out.print("("+i+","+j+") ");
      }
    System.out.println();
}
```

Chapter 04 : Methods

01*Why using methods?

A Method is a collection of statements that are grouped together to perform an operation
Using methods in Java helps organize code into reusable , manageable blocks , making programs easier to read , test , and maintain

02*Opening Problem with a method with a return value:

Opening Problem:

```
int sum = 0;
for(int i=0; i<=10 ; i++) {
   sum+=i;
   System.out.println("Sum from 1 to 10 is " +sum);
  }
int sum = 0;
for(int i=20; i<=30 ; i++) {
     sum+=i;
     System.out.println("Sum from 20 to 30 is " +sum);
  }
int sum = 0;
for(int i=35; i<=10 ; i++) {
   sum+=i;
   System.out.println("Sum from 35 to 45 is " +sum);
}
```

Solution:

```
public static int sumRange(int i1,int i2) {
  int sum=0;
  for(int i=i1 ; i<=i2 ; i++) {
    sum+=i;
    return sum;
}
```

Invoke & Calling the Method:

```
public static void main(String[] args) {
System.out.println("Sum from 1 to 10 is "+sumRange(1,10)); //here it called arguments
System.out.println("Sum from 20 to 30 is "+ sumRange(20,30));
System.out.println("Sum from 35 to 45 is " + sumRange(35,45));
  }
```

```
Output
Sum from 1 to 10 is 55
Sum from 20 to 30 is 275
Sum from 35 to 45 is 440
```

03*Void Method:

```
public class ProcedureSumRange {
public static void sumRange(int i1,int i2) {
  int sum = 0;
  for(int i=i1 ; i<=i2 ; i++) {
    int sum+=i;
    System.out.println("Sum: "+sum);
  }
public static void main(String[] args) {
    sumRange(1,10);
    sumRange(20,30);
    sumRange(35,45);
   }
}
```

Output
Sum from 1 to 10 is 55 Sum from 20 to 30 is 275 Sum from 35 to 45 to 440

04*Method Signature:

Function Method:

```
public  static returnValueType methodName(parameters) {
  //code
  return value;
}
```

Procedure Method:

```
public static void methodName(parameters) {
    //code
  }
```

Chapter 05 : Arrays

01*Types of Arrays:

-->Single-Dimensional Array
-->Multidimensional Array

02*Single Dimensional Array:

A single-dimensional array is a collection of elements of the same data type stored in a single row
datatype[] variable = new Datatype[size];
datatype[] variable = {a,b,c,d};

```
public class Main {
   public static void main(String[] args) {
      int[] numbers = {10, 20, 30, 40, 50};
      for (int i = 0; i < numbers.length; i++) {
         System.out.println(numbers[i]);
      }
   }
}
```

Output
10
20
30
40
50

03*Multidimensional Array:

A multidimensional array is an array that contains other arrays.The most common type is a 2D array(rows and columns)

datatype[][] variable = new datatype[rowsize][columnsize];

```
Datatype[][] variable = { {a,b,c} ,
                          {d,e,f} ,
                          {g,h,i}
                        };
```

```
public class Main {
   public static void main(String[] args) {
      int[][] matrix = {
         {1, 2, 3},
         {4, 5, 6},
         {7, 8, 9}
      };
      for (int row = 0; row < matrix.length; row++) {
         for (int col = 0; col < matrix[row].length; col++) {
            System.out.print(matrix[row][col] + " ");
         }
         System.out.println();
      }
   }
}
```

Output
1 2 3 4 5 6 7 8 9

Chapter 06 : Classes and Objects:

01*Definitions:

Classes:

A class is a blueprint or template used to create objects.
It defines the attributes (data) and methods (behavior) of objects.

Objects:

An object is an instance of a class.
It contains its own data and can perform actions defined by the class.

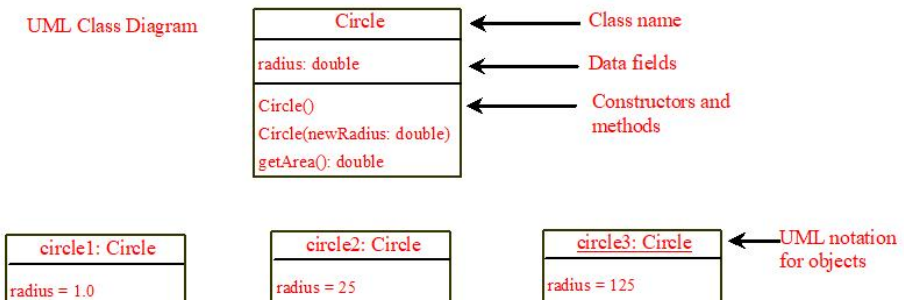
Constructor:

A constructor is a special method in a class used to initialize objects . It is called automatically when an object is created and often assigns initial values to the class fields

Garbage Collection:

Garbage Collection is the automatic process of removing unused objects from memory.
It helps manage memory and prevents memory leaks in Java.

UML Class Diagram:



02*Creating an object:

```
className objectName = new className();
```

Or the other way is to write:

```
className objectName; //declaration statement
objectName = new className ; //creation statement
```

Example:

```
Circle myCircle1 = new Circle();
Circle myCircle2 = new Circle(5.0);
```

03*Accessing an Object's Data and Methods:

To reference the object's data we use the dot operator between the reference and the data field:

```
objectName.dataFieldName;
```

To invoke/call an object's method we use the dot operator between the reference and the method:

```
objectName.methodName(arguments);
```

04*Reference Data Fields and the null Value:

```
class Circle {
   int radius; // Default value is 0
   double area; // Default value is 0.0
   boolean isFilled; //Default value is false
   String color; //Default value is null
}
```

Hence null is the default value for any data field of a reference type
Note that null is a Keyword in Java

05*Using Classes from the Java Library:

The Date Class:

The Date class is found in the util package in the Java library To use we write

```
import java.util.Date;
public class Main {
   public static void main(String[] args) {
     Date date = new Date();
     System.out.println(date.toString());
     }
}
   //Output: it displays a string like Mon Mar 23 20:50:19 EST 2020
```

The + sign indicates public modifer →

java.util.Date	
+Date()	Constructs a Date object for the current time.
+Date(elapseTime: long)	Constructs a Date object for a given time in milliseconds elapsed since January 1, 1970, GMT.
+toString(): String	Returns a string representing the date and time.
+getTime(): long	Returns the number of milliseconds since January 1, 1970, GMT.
+setTime(elapseTime: long): void	Sets a new elapse time in the object.

The Random Class:
The Random class found also in the util package is used to generate random values of different types

```
import java.util.Random();
Random r = new Random();
int x = r.nextInt(); // x could be any integer value
int y = r.nextInt(10); // y could be any integer value between 0 and 9
double z = r.nextDouble(); // z could be any double value between 0.0 inclusive and 10 exclusive
boolean flag = r.nextBoolean(); //flag could be either false or true
```

java.util.Random	
+Random()	Constructs a Random object with the current time as its seed.
+Random(seed: long)	Constructs a Random object with a specified seed.
+nextInt(): int	Returns a random int value.
+nextInt(n: int): int	Returns a random int value between 0 and n (exclusive).
+nextLong(): long	Returns a random long value.
+nextDouble(): double	Returns a random double value between 0.0 and 1.0 (exclusive).
+nextFloat(): float	Returns a random float value between 0.0F and 1.0F (exclusive).
+nextBoolean(): boolean	Returns a random boolean value.

06*Accessing Static vs Non-Static Attributes:

Static Attributes:
Static attributes belong to the class itself , not to individual objects
You can access static attributes directly using the class name without creating an object
Syntax: className.attribute;
Shared: All objects share the same static attribute value

Example:

```
class Circle {
   static double radius = 10.0;
 }
class Main {
   public static void main(String[] args) {
     System.out.println("Static Radius: "+myCircle.radius);
   }
}
```

Non-Static Attributes:
Non-static attributes belong to individual objects of the class
To access non-static attributes , you must create an object of the class
Syntax: objectName.dataFieldName;
Unique: Each object has its own copy of the non-static attribute

Example:

```
class Circle {
   double radius;
 }
class Main {
   public static void main(String[] args) {
   Circle myCircle1 = new Circle();
   myCircle.radius = 5;
   System.out.println("Non-static Radius: "+myCircle.radius);
   }
}
```

Static Constants:

Static Constants are final variables shared by all the instances of the class

Constants should be declared as a static final . For example , the constant PI in the Math class is defined as: static final double PI = 3.141592653589799323846;

Example:

```
public class Circle {
   double radius; //instance data (each Circle object has it own radius)
   static int numberOfObjects = 0; //static data(shared by all Circle objects)
   public Circle(double r) {
     radius= r;
     static int numberOfObjects = 0 ; //static data (shared by all Circle objects)
  }
   public Circle(double r) {
     radius = r;
     numberOfObjects++; // increase by 1 each time an object is created
  }
   public double getArea() { // instance method
     return radius*radius*Math.PI;
  }

   public static int getNumberOfObjects() { //static method
     return numberOfObjects;
    }
  } //end class Circle

class TestCircle {
   public static void main(String[] args) {
     System.out.println("Before creating objects");
     System.out.println("The number of Circle objects is "+ Circle.numberOfObjects);
     Circle c1 = new Circle(1);
     System.out.println("After creating c1");
     System.out.println("The number of Circle objects is " + c1.numberOfObjects);
     System.out.println("The number of Circle objects is " + Circle.numberOfObjects);
     Circle c2 = new Circle(5);
     System.out.println("After creating c2");
     System.out.println("The number of Circle object is " + c1.numberOfObjects);
     System.out.println("The number of Circle object is " + c2.numberOfObjects);
     System.out.println("The number of Circle object is " + Circle.numberOfObjects);
    }
  }
```

Output
Before creating objects The number of Circle objects is 0 After creating c1 The number of Circle objects is 1 The number of Circle object is 1 After creating c2 The number of Circle objects is 2 The number of Circle objects is 2 The number of Circle objects is 2

07*Array of Objects:

className[] arrayName = new className[arraySize]; //Declare and Create the array
arrayName[index] = new className(parameters); //intialize each element

Example Using Array of Circles:

```
import java.util.Random();
public class TestCircle {
   public static void main(String[] args) {
     Random r = new Random();
     Circle[] circleArray;
     for(int i=0;i<circleArray.length ; i++) {
       circleArray[i] = new Circle(r.nextDouble()*10);
       System.out.print("Circle " + i + "radius = " + circleArray[i].getRadius());
       System.out.println("area = " + circleArray[i].getArea());
      }
   }
}
```

Output

```
Circle 0 radius = 5.323631308614646      area = 89.03603544910978
Circle 1 radius = 6.964428451073555      area = 152.37749675835462
Circle 2 radius = 3.6253233159062823     area = 41.28985531182978
Circle 3 radius = 7.25542582457795       area = 165.37721943564716
Circle 4 radius = 6.559096414885918      area = 135.15680048633092
Circle 5 radius = 5.8449836756578595     area = 107.32885044286944
Circle 6 radius = 7.4077945641189435     area = 172.39621729029602
Circle 7 radius = 7.515333962633156      area = 177.43792141377583
Circle 8 radius = 0.5383713110649391     area = 0.9105707398934727
Circle 9 radius = 0.9508741997827064     area = 2.8405078920179796
```

08*Visibility Modifiers:

Public : The member is accessible from anywhere in the program (within the same class + package, or from other packages)
Private : The member is accessible only within the same class where it is defined
Default : The member is accessible only within the same package
Note: public and private are Keywords

09*Data Field Encapsulation:

Why should data fields be private?
.To protect the integrity of the data by restricting direct access and modification
.Prevents unintended or invalid changes to the data fields from outside the class

The Need for Get and Set Methods:
Getter methods allow controlled , read-only access to private data fields
Setter methods allow controlled modification of private data fields , often including validation logic to ensure the data remains valid

```
class Circle {
  private double radius;
  public Circle(double newRadius) {
    if(newRadius >=0)
      radius = newRadius ;
    else
      radius = 0;
    }
  public double getRadius() {
    return radius;
   }
  public void setRadius(double newRadius) {
```

```
    if(newRadius >=0)
      radius = newRadius;
    else
      radius = 0;
   }
  public double getArea() {
     return radius*radius*Math.PI;
   }
}
```

10*Methods to Output The Data Fields:

The print() Method:

```
public void print() {
  System.out.println("The radius is " + radius);
  System.out.println("The number of objects is " + numberOfObjects);
}
```

The toString() Method:

```
public String toString() {
  String s = "The radius is " + radius + "\nThe number of objects is " + numberOfObjects;
}
```

Or simply

```
public String toString() {
  return "The radius is " + radius + "\nThe number of objects is " + numberOfObjects;
}
```

Calling the toString() method:

```
public class TestCircle {
  public static void main(String[] args) {
    Circle c1 = new Circle(5.0) ;
    System.out.println(c1.toString());
    Circle c2 = new Circle(2.0);
    System.out.println(c2.toString());
      }
   }
```

```
Output
The radius is 5.0
The number of objects is 1
The radius is 2.0
The number of objects is 2
```

11*Passing Objects to Methods:

Passing by Reference : Instead of copying the object , the memory address is sent to the method . Therefore , changes made to the object inside the method are reflected on the original object.

Example:

```
class Circle {
     private double radius;
     public Circle(double r) {
       radius = r;
       }
     public double getRadius() {
       return radius;
       }
     public void setRadius(double r) {
       if(r>0)
         radius = r;
       else
         System.out.println("Invalid radius);
```

```
    }
  }
  public class Main {
   public static void main(String[] args) {
     Circle myCircle = new Circle(5.0);
     System.out.println("Before : " + myCircle.getRadius());
     changeRadius(myCircle);
     System.out.println("After : " + myCircle.getRadius());
    }
    public static void changeRadius(Circle c) {
     c.setRadius(10.0);
     }
  }
```

Output
Before : 5.0 After : 10.0

12*The Scope of a Variable:

Local Scope :

Declared inside methods , constructors ,or blocks , accessible only within that block

Instance Scope :

Declared in a class but outside methods , belongs to an object , and accessible by all methods of the class

Class /Static Scope:

Declared as static in a class , shared across all objects , and accessed without creating an object

```
public class ScopeExample {
  int instanceVar = 10; //Instance scope
  static int staticVar = 20; //Static scope
  public void show() {
    int localVar = 30; //Local scope
    }
}
```

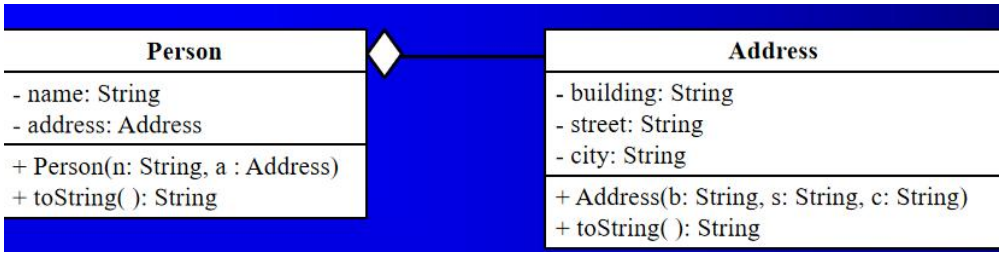
What if we do not use the this Keyword when we should?

```
public class Foo {
  private int i = 5;

  public void setI(int i) {
          i = i;
  }
}
```

parameter parameter

```
Suppose that f1 is an object of Foo.

Invoking f1.setI(10) will not initialize the
data field i to 10. It will instead set the
parameter i to itself (i.e. change the
parameter i from 10 to 10).
```

13*Object Composition:




```
class Person {
  private String name;
  private Address address; //This is the statement that creates the composition relationship
  public String toString() {
    return "name = " + name + " address = " + address.toString();
    }
}
class Address {
  private String building;
  private String street;
  private String city;
}
  public Address(String b,String s , String c) {
    building = b;
    street = s;
    city = c;
  }
  public String toString() {
    return building + ", " + street + ", " + city;
  }
}
public class Test {
  public static void main(String[] args) {
    Address a = new Address("Building A" , "Hamra Stree" , "Beirut");
    Person p = new Person("Jawad" , a);
    System.out.println(p);
    }
}
```

14*Enhanced For Loop (for-each):

The enhanced for loop is a simplified version of the for loop to iterate through arrays or collections.
"For each element of this array/collection , do the following."

Syntax:

```
for(Type element :  collectionOrArray) {
 //use element
}
```

Traditional for loop:

```
int[] arr = {1 ,2 , 3 ,4};
for(int i = 0;  i<arr.length ; i++) {
  System.out.println(arr[i]);
}
```

Enhanced for-each loop:

```
int[] arr  = {1,2,3,4};
for(int num: arr) {
  System.out.println(num);
  }
```

```
Output
1
2
3
4
```

Examples with collections:

```
import java.util.ArrayList;
public class TestForEach {
  public static void main(String[] args) {
    ArrayList<String> names = new ArrayList<>();
    names.add("Ali");
    names.add("Sara");
    names.add("John");
    for(String name : names) {
      System.out.println(name) ;
      }
   }
}
```

```
Output
Ali
Sara
John
```

Example : iterating over an Array of Objects

```
class Student {
  String name;
  int age;
  Student(String name , int age) {
    this.name = name;
    this.age = age;
    }
  void display() {
    System.out.println(name + " is " + age + " years old");
   }
}
public class TestForEachObject {
    public static void main(String[] args) {
      Student[] students = {
        new Student("Ali",20) ,
        new Student("Sara" , 22) ,
        new Student("John" , 19);
  };
      //Enhanced for loop (for-each) over objects
      for (Student s : students) {
        s.display() ;  //call method on each object
        }
    }
}
```

```
Output:
Ali is 20 years old
Sara is 22 years old
John is 19 years old
```

Example: Iterating over a Collection of Objects

```
import java.util.ArrayList;
public class TestForEachObjectList {
  public static void main(String[] args) {
    ArrayList<Student> students = new ArrayList<>();
    students.add(new Student("Ali" , 20));
    students.add(new Student("Sara" ,22));
    students.add(new Student("John" , 19));
    //Enhanced for loop over ArrayList of objects
    for(Student s : students) {
      s.display();
     }
  }
}
```

Output
Ali is 20 years old Sara is 22 years old John is 19 years old

Chapter 07 : Strings

01*Constructing a String:

String :

Sequence or Characters

Using the constructor:

```
String s = new String("Text");
String s = new String("Hello World");
```

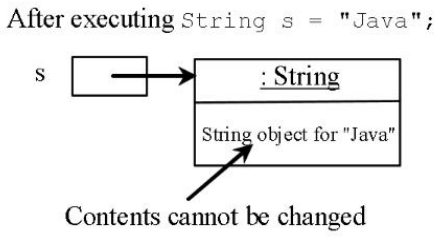
Using a String literal:

```
String s = "Text";
String s = "Hello World";
```

Using an array of characters:

The following statement creates the String "Jawad";

```
char k = { 'J' , 'a' , 'w' , 'a' , 'd' };
String text = new String(k);
```



02*Internal Strings:

JVM uses a unique instance for string literals with the same character sequence to improve efficiency and save memory . Such an instance is called an interned string . For example , the following statements :

```
String s1 = “Welcome to Java” ;
String s2 = new String(“Welcome to Java”);
String s3 = “Welcome to Java”;
System.out.println(“s1 == s2 is ” + (s1 == s2));   //s1 == s2 is false
System.out.println(“s1 == s3 is “ + (s1 == s3));  //s1 == s3 is false
```

03*String Comparison:

equals() : is used to compare the compare the contents of strings

```
String s1 = new String(“Welcome”);
String s2 = “Welcome”;
String s3 = “Hi”;
if(s1.equals(s2)) {
   //checks if s1 and s2 have the same contents
   //For the above example it gives true
  }
if(s1.equals(s3)) {
   //checks if s1 and s3 have the same contents
   //For the above example it gives false
  }
```

== : is used to compare the string references

```
String s1 = new String(“Welcome”);
String s2 = “Welcome” ;
String s3 = s1;
if(s1 == s2) {
   //checks if s1 and s2 have the same reference
   //For the above example it gives false
  }
if(s1 == s3) {
   //checks if s1 and s3 have the same reference
   //For the above example it gives true
  }
```

equalsIgnoreCase() : Ignores the letter case (lower versus upper case) when comparing

```
String s1 = new String(“Welcome”);
String s2 = “Welcome” ;
if(s1.equals(s2)
   //gives false since w is different from W
if(s1.equalsIgnoreCase(s2))
   //gives true since w and W are considered the same
```

compareTo() : Returns an integer greater than 0 , equal to 0 , or less than 0 to indicate whether this string is greater than , equal to , or less than s1

```
String s1 = new String(“Welcome”);
String s2 = “Welcome” ;
if(s1.compareTo(s2) >0) //is s1 greater than s2?
if(s1.compareTo(s2) >0) //is s1 smaller than s2?
if(s1.compareTo(s2) == 0) // do s1 and s2 have the same content?
String s1 = “abc” , s2 = “abc” ;
String s3 = “aba” , s4 = “abd” ;
int x;
x = s1.compareTo(s2);
//x =0 since s1 and s2 are equal
x=s1.compareTo(s3);
// x>0 sinxe s1>s3 because ‘c’ is > ‘a’ (‘c’  = 99 & ‘a’ = 97)
// See ASCII Table
```

```
x = s1.compareTo(s4);
//x<0 since s1<s4 because 'c' is <  'd' ( 'c' = 99 & 'd'=100)
```

compareToIgnoreCase(): Same as compareTo except that the comparison is case-insensitive

```
String s1 = "abc" , s2 = "ABC" ;
int x;
x = s1.compareTo(s2);
//x>0 since s1>s2 because 'a' > 'A' ('a' = 97 , 'A' = 65)
x = s1.compareToIgnoreCase(s2) ;
//x =0 since s1 and s2 are equal when the case is ignored
```

religionMatches() : This method compares portions of two strings for equality

```
String s1 = "scde" , s2 = "abcdef";
if(s1.regionMatches(1,s2,2,3)
/*1 is the starting index in s1, and 2 is the starting index in s2 , 3 is the length or the number of characters to be compared*/
```

startsWith(prefix) : to check whether string starts with a specified prefix

endsWith(suffix) : to check whether string ends with a specified suffix

```
String s = "howtodoinJava.com";
if(s.startWith("how")) //gives true
if(s.startWith("howl")) //gives false
if(s.endsWith("com")) //gives true
if(s.endsWith("dom")) //gives false
```

04*String length , Characters , and Combining Strings:

length() : Returns the number of characters in this string

```
String p = "Hello World";
int len = p.length(); //len = 11
```

charAt() : Returns the character at the specified indwx from this string

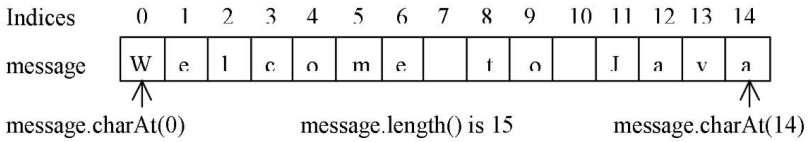
```
String m = "Hello";
for(int i=0;i<m.length();i++)
System.out.println(m.charAt(i));
```

```
Output
H
e
l
l
o
```

concat() : Returns a new string that concatenate this string with string s1

```
String s1 = "Welcome" , s2 = " to Java";
String s3 = s1.concat(s2);
s3 will contain the string "Welcome to Java"
```

05*Obtaining Substrings:



substring(beginIndex) : Returns this string's substring that begins with the character at the specified begin Index and extends to the end of the string

```
String s1 = "Welcome to Java" ;
```

To extract the substring from index 11 till the end we write:

```
String s2 = s1.substring(11);
```

substring(beginIndex , endIndex) : Returns the string's substring that begins at the specified beginIndex and extends to the character at the index endIndex -1

```
String s1 = "Welcome to Java";
String s1 = s1.substring(0,11); // Welcome to
```

06*Converting , Replacing , and Splitting Strings:

toLowerCase() : Returns a new string with all characters converted to lowercase .

```
"Welcome".toLowerCase() returns a new string , welcome
```

toUpperCase() : Returns a new string with all character converted to uppercase.

```
"Welcome".toUpperCase() returns a new string , WELCOME
```

trim() : Returns a new string with blank characters trimmed on both sides.

```
" Welcome to Java    ".trim() returns a new string, Welcome to Java
```

replace() : Returns a new string that that replaces all matching character in this string with the new character.

```
"Welcome".replace('e' ,'A') returns a new String . WAlcomA
```

replaceFirst() : Return a new string that replaces the first matching substring in this string with the new substring.

```
"Welcome".replace("e","AB") returns a new String. WABlcome
```

replaceAll() : Returns a new string that replaces all matching substrings in this string with the new substring.

```
"Welcome".replaceAll("e","AB") returns a new String , WABlcomAB
```

replace(oldString , newString) : Works exactly like replaceAll.

```
"Welcome".replace("e","AB") returns a new String , WABlcomAB
```

07*Spliting a String:

The split method can be used to extract tokens from a string with the specified delimiters . For example , in the following code the delimiter is the string "#"

split(delimiter) : Returns an array of strings consisting of the substrings split by the delimiter.

```
String s = "Java#HTML#Perl" ;
String[] tokens = s.split("#");
For(int i=0; I<tokens.length ;i++)
System.out.print(tokens[i]+" ");
```

The delimiter # is not stored in the array tokens(an array of strings)

```
String s = "cdabefabgh" ;
String[] tokens = s.split("ab");
For(int i=0 ;I<tokens.length;i++)
System.out.print(tokens[i] + " ");
```

08*The "." delimiter:

If want to split a string using the delimiter dot "." such as

```
String a = "a.jpg";
String[] str = a.split(".");
```

The split does not work because "." is a reserved character
Instead, we should use the following statement:

```
String[] str = a.split("\\.");
```

09*Finding a Character or a substring in a String:

indexOf(ch : char) : int Returns the index of the first occurrence of ch in the string. Returns -1 if not matches

```
"Welcome to Java".indexOf('W') returns 0
```

indexOf(ch: char , fromIndex: int): int Returns the index of the first occurrence of ch after fromIndex in the string. Returns -1 if not matches

```
"Welcome to Java".indexOf('x') returns -1
```

indexOf(s: String): int Returns the index of the first occurrence of string s in this string. Returns -1 if not matches

```
"Welcome to Java".indexOf('o' , 5) returns 9
```

indexOf(s : String , fromIndex: int): int Returns the index of the first occurrence of string s in this string after fromIndex.. Returns -1 if not matches

```
"Welcome to Java".indexOf("come") returns 3
```

lastIndexOf(ch : int): int Returns the index of the last occurrence of ch in this string. Returns -1 if not matches

```
"Welcome to Java".indexOf("Java" , 5) returns 11
```

lastIndexOfch : int , fromIndex : int) : int Returns the index of the last occurrence of ch before fromIndex in this string. Returns -1 if not matches

```
"Welcome to Java".indexOf("java" , 5) returns -1
```

lastIndexOf(s : String) : int Returns the index of the last occurrence of string s . Returns -1 if not matches

```
"Welcome to Java".lastIndexOf('a') returns 14
```

valueOf(c: char) : String Returns a string considering of the character C

```
char c = 'A' ;
String s = String.valueOf(c);
System.out.println(s);
```

```
Output
A
```

valueOf(data : char[]) : String Returns a string considering of the characters in the array

```
char[] data = {'J' , 'a' , 'v' , 'a'};
String s = String.valueOf(data);
System.out.println(s)
```

```
Output
Java
```

valueOf(d : double) : String Returns a string representing the double value

```
double d = 3.14;
String s = valueOf(d);
System.out.println(s)
```

Output
3.14

valueOf(f : float) : String Returns a string representing the float value

```
float f = 2.5f;
String s = String.valueOf(f);
System.out.println(s)
```

Output
2.5

valueOf(i : int) : String Returns a string representing the long value

```
int i =10;
String s = String.valueOf(I);
System.out.println(s);
```

Output
10

valueOf(b : boolean) : String Returns a string representing the boolean value

```
boolean b = true;
String s = String.valueOf(b);
System.out.println(s);
```

Output
true

10*Converting Strings to Integer and Double using Integer.parseInt() and Double.parseDouble() :

To convert a String into an integer we use the method Integer.parseInt() and into a double we use the method Double.parseDouble().

```
String s1 = “12” , s2 = “12.5” ;
int n = Integer.parseInt(s1);
double m = Double.parseDouble(s2);
```

As a result n will contain the integer value 12 and m will contain the double value 12.5
Note that if s1 and s2 contain non digit characters then an error will occur. Of course s2 can contain the dot .

Chapter 08 : Inheritance and Polymorphism

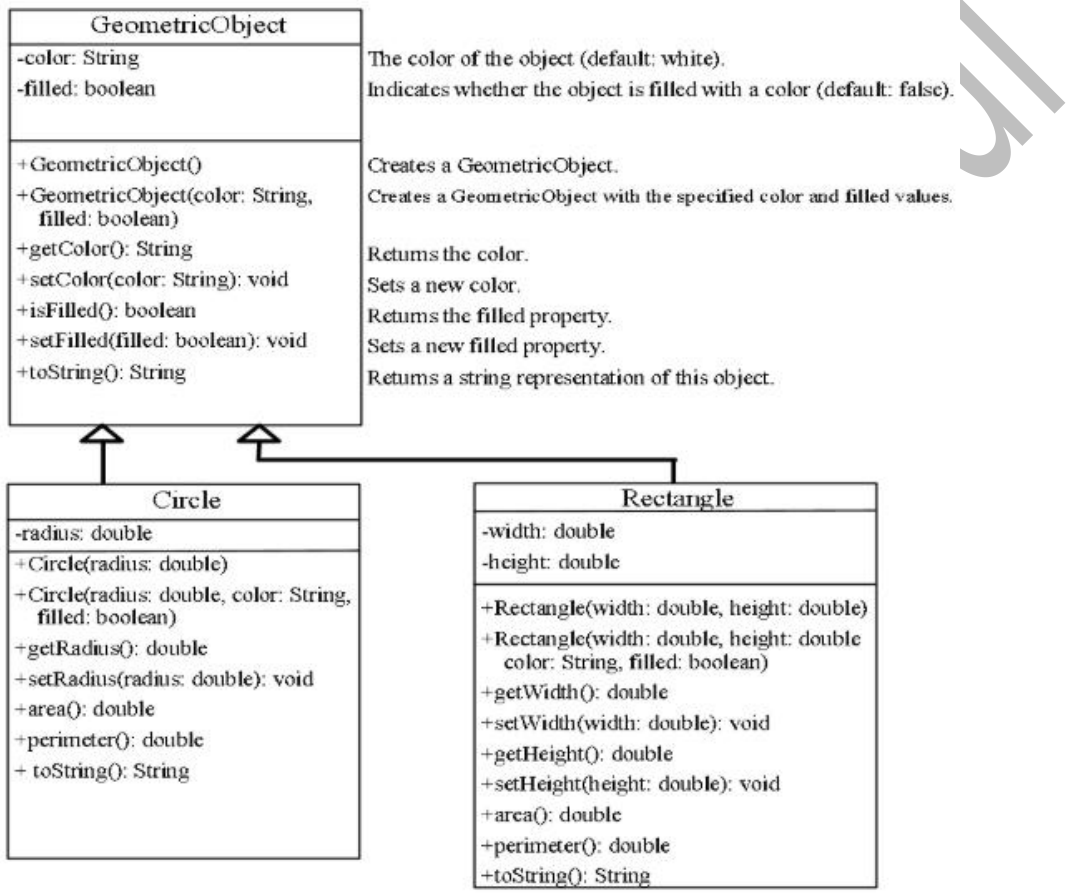
01*Inheritance Definition:

Inheritance: is a mechanism where one class(child/subclass) acquires the properties and behaviors of another class(parent/superclass)

02*Super classes and Subclasses:

A superclass is the parent class that provides its properties and methods to other classes , serving as a base for inheritance . A subclass is the child class that inherits the features of the superclass and can also define its own additional properties and behaviors

03*Geometric Object Class Application:



```
class GeometricObject {
  private String color;
  private boolean filled;
  public GeometricObject() {
    color = “white”;
    filled = false;
    }
  public GeometricObject(String c , boolean f) {
    color = c;
    filled = f;
  }
public String getColor() {
  return color;
}
public void setColor(String c) {
  color = c;
}
public boolean isFilled() {
  return filled;
}
public void setFilled(boolean f) {
  filled = f;
}
public String toString() {
  String s = “The color is ”+ color;
  if(filled)
    s+= “.\nThe geometric object is filled”;
  else
    s+= “.\The geometric object is not fillled”;
  return s;
  }
} //end class GeometricObject

class Circle extends GeometricObject {
  private double radius;
  public Circle(String c , boolean f , double r) {
    super(c,f); //calls the constructor of GeometricObject using the keyword super
    radius = r;
  }
public double getRadius() {
  return radius;
}
public void setRadius(double radius) {
  this.radius = radius;
}
public double area() {
  return radius*radius*Math.PI;
}
public double perimeter() {
  return 2 * radius *Math.PI;
}
public String toString() {
  //call toString of GeometricObject using the keyword super
  return super.toString() + “.\nThe radius is ”+radius;
   }
} //end class circle
class Rectange extends GeometricObject {
  private double width;
  private double height;
  public Rectange(String c , boolean f , double width , double height) {
    super(c,f); //calls constructor of GeometricObject
    this.width = width;
    this.height = height;
```

```
  }
  public double getWidth() {
    return width;
}
  public void setWidth(double width) {
    this.width = width;
   }
public double getHeight() {
  return height;
}
public void setHeight(double height) {
  this.height = height ;
}
public double area() {
  return width*height;
}
public double perimeter() {
  return 2 * (width+height);
}
public String toString() {
//calls toString of GeometricObject
return super.toString() + "\.nThe height is " + height + ".\nThe width is " + width;
   }
} //end class Rectangle
public class TestingInheritance {
  public static void main(String[] args) {
    Circle c1 = new Circle("red" , true ,2);
    System.out.println(c1.getColor());
    System.out.println(c1.getRadius());
    System.out.println(c1);
    GeometricObject  g1  = new GeometricObject("black" , false);
    System.out.println(g1.getColor());
    //System.out.println(g1.getRadius();
        // getRadius() will give an error since class GeometricObject does not have a method called getRadius()
    System.out.println(g1);
    Rectangle r1 = new Rectange("green" , true , 3 ,4);
    System.out.println(r1.getColor());
    System.out.println(r1.getHeight());
    System.out.println(r1);
    }
}
```

04*Using the super Keyword:

The super keyword is used to call a superclass's constructor or access its methods and fields
It can be used in two ways:
-> To call superclass constructor
-> To call a superclass method

super() :
Invokes the no-arg constructor of its superclass
super(arguments):
Invokes the super class constructor that matches the arguments

05*Example on the Implact of a Super class without no-arg constructor:

What is the error ?

```
public class Fruit {
   System.out.println("Fruit's constructor is invoked");
   }
}
public class Apple extends Fruit {
   public Apple(String name) {
    }
}
```

The constructor of Apple does not contain an explicit call to the superclass Fruit constructor so it will call the no-arg constructor of Fruit which does not exist.This will cause an error

06*Overriding Methods:

Method overriding is when a subclass provides a specific implementation of a method that is already defined in its superclass , with the same method signature

An instance(non-static) method can be overridden only if it is accessible . Thus a private method cannot be overriden , because it is not accessible outside its own class

Like an instance method , a static method can be inherited . However , a static method cannot be overridden .

```
class Circle extends GeometricObject {
  //Other methods are omitted
  @override
  public String toString() {
    return super.toString() + "/nradius is " + radius;
    }
}
```

Example:

```
class Animal {
public void sound() {
   System.out.println("Animal makes a sound");
   }
}
class Dog extends Animal {
@override
public void sound() {
   System.out.println("Dogs barks) ;
   }
}
public class Main {
  public static void main(String[] args) {
     Animal myAnimal = new Animal() ;
     Animal myDog = new Animal() ;
     myAnimal.sound() ;
     myDog.sound();
   }
}
```

07*Overloading Methods:

Method overloading is the ability to define multiple methods with the same name but different parameter lists(number or type) within the same class

A.If the type of the parameters is different:

```
public static int max(int num1 , int num2) {
   //body
  }
public static double max(int num1 , int num2) {
   //body
  }
```

B.If the number of the parameters is different:

```
public static int sum(int num1 , int num2) {
  //body
}
public static int sum(int num1 , int num2 , int num3) {
  //body
}
```

C.If the order of the parameters is different:

```
public static void method(int num , char ch) {
   //body
  }
public static void method(char ch , int num) {
   //body
  }
```

What if we said?

```
public static int max(int n1 , int n2) {
   //body
  }
public static double max(int n1 , int n2) {
   //body
}
```

The code you provided will result in a compilation error because method overloading requires methods to differ in parameters , but not just in return type

08*The object class and its toString() Method:

.The object class itself is found in the java.lang package , and every class in Java implicity extends it unless it specifically extends another class

.Every class in Java is a subclass of the class Object found in the java library in the lang package

If no inheritance is specified when a class defined , the superclass of the class is Object by default . For example , the following two class definitions are the same

```
public class ClassName {
  ...
}
```

Equivalent

```
public class ClassName extends Object {
  ...
}
```

09*Polymorphism:

Polymorphism is one the core concepts of Object-Oriented Programming(OOP) . It allows an object to take on many forms . In Java , it means that a single reference variable of a superclass type can refer to objects of different subclass types , and the appropriate method is called at runtime

Example:

```
class Animal {
  void sound() {
     System.out.println("Some generic animal sound");
    }
}
class Dog extends Animals {
  void sound() {
     System.out.println("Woof");
    }
}
class Cat extends Animal {
   void sound() {
     System.out.println("Meow!");
```

```
    }
}
public class Main {
    public static void main(String[] args) {
    Animal a1 = new Dog() ;
    Animal a2 = new Cat();
    a1.sound(); // Woof!
    a2.sound(); //Meow!
        }
  }
```

10*Casting Objects:

Means converting one type of reference to another within the same inheritance hierarchy , allows us to treat an object as a more specific or more general type

Example:

```
Animal a = new Dog(); //Upcasting
((Dog)a).sound() ; //Downcasting to call Dog's specific method
Animal a = new Dog() ; //Upcasting
Dog d = (Dog) a; //Downcasting (valid)
d.sound();
```

The instanceof Operator:

The instanceof operator checks whether an object is an instance of a specific class or subclass
It helps prevent ClassCastException during downcasting

Example:

```
Animal a = new Cat();
if(a instanceof Dog) {
    Dog d = (Dog)a;
    d.sound();
  }
else {
    System.out.println("Not a Dog object");
}
```

Output
Not a Dog object

Chapter 09 : ArrayList

01*The ArrayList class:

The ArrayList class is a resizable array from the java.util package , which allows dynamic addition and removal of elements . Unlike regular arrays , its size adjusts automatically as elements are added or removed , we **import java.util.ArrayList;**

02*The ArrayList Methods:

ArrayList() : Creates an empty ArrayList

```
ArrayList<String> names = new ArrayList<>();
System.out.println(names);
```

Output
[]

add(): Adds an element to the list

Adds an element to the list

```
names.add("Ali");
names.add("Sara");
System.out.println(names);
```

Output
[Ali,Sara]

clear(): Removes all elements

```
names.clear();
System.out.println(names);
```

Output
[]

contains() : Checks if element exists

```
System.out.println(names.contains("Ali"));
System.out.println(names.contains("Khaled"));
```

Output
true fasle

get() : Returns element at index

```
System.out.println(names.get[1]);
```

Output
Sara

indexOf() : First occurrence index

```
System.out.println(names.indexOf("Ali"));
```

Output
0

lastIndexOf():Last occurrence index

```
System.out.println(names.lastIndexOf("Ali"));
```

Output
3

remove(int index) : Removes element by index

```
names.remove(2);
System.out.println(names);
```

Output
[Ali , Sara ,Ali]

remove(object o) : Removes first matching element

```
names.remove("Ali");
System.out.println(names) ;
```

Output
[Sara , Omar]

size(): Returns number of elements

```
System.out.println(name.size());
```

Output
4

set(): Replaces element at index

```
names.set(1,"Lina");
System.out.println(name);
```

Output
[Ali , Lina , Omar , Ali]

03*The ArrayList class as a generic class:

ArrayList<Type> list = new ArrayList<>();

Examples:

```
ArrayList<String> cities = new ArrayList<String>();
ArrayList<Date> dates = new ArrayList<Date>();
```

04*Non-Generic versus Generic ArrayList:

Generic ArrayList:

Can store only one type of objects(specified using <Type>

Type-safe ? Then no Casting Needed

Errors are caught at compile-time

Example:

```
import java.util.ArrayList;
public class Main {
  public static void main(String[] args) {
    ArrayList<String> list = new ArrayList<>() ; //Generic
    list.add("Java");
    list.add("Python");
    //list.add(10); will cause a compile-error time
    String s = list.get(0);
    System.out.println(s);
     }
  }
```

Output
Java

Non-Generic ArrayList :

Can store any type of objects

Not type-safe -> you need casting when retrieving elements

Errors are caught at runtime , not compile-time

Example:

```
import java.util.ArrayList ;
public class Main {
  public static void main(String[] args) {
    ArrayList list = new ArrayList(); //Non-generic
    list.add("Java");
    list.add(10);
    list.add(3.5);
    String s = (String) list.get() ;
    System.out.println(s);
    //String x = (String) list.get(1); //Runtime error
    }
}
```

Output
Java

Chapter 10 : Exception Handling and Text I/O :

01*Introduction:

When a program runs into a runtime error(also known as exception) , the program terminates abnormally . How can you handle the runtime error so that the program can continue to run or terminate gracefully ? this is the subject we will introduce in this chapter

02*Exception Handling Overview:

Without Exception Handling:

```
public Main {
  public static void main(String[] args) {
    int a = 10 ;
    int b = 0;
    int result = a/b;
    System.out.println("Result: " + result);
   }
}
```

Output:
Exception in thread "main" java.lang.ArithmeticException: /by zero at Main.main(Main.java: 5)

Without Exception Handling:

```
public class Main {
  public static void main(String[] args) {
    try {
      int a = 10;
      int b = 0;
      int result = a/b;
      System.out.println("Result "+ result) ;
    }
    catch(ArithmeticException e) {
      System.out.println("Error: Cannot divide by zero.");
      }
    }
  }
```

Output
Error: Cannot divide by zero

Fixing it without using Exception Handling:

```
public class Main {
  public static void main(String[] args) {
    int a = 10;
    int b = 0;
    if(b!=0) {
      int result = a/b;
      System.out.println("The quotient is: " + quotient);
    }
    else {
      System.out.println("Error: Cannot divide by zero");
       }
    }
}
```

Output
Error: Cannot divide by zero

03*Throwing and catching an Exception:

What is try-catch?

The try-catch block in Java is a mechanism to handle exceptions(runtime errors)
Try block: Contains the code that might throw an exception
Catch block: Catches and handles the exception if it occurs , preventing the program from crashing

What is the Value Thrown ?

When an exception occurs , Java creates an exception object that contains:
3.2.1.The type exception (e.g. , ArithmeticException , NullPointerException)
3.2.2.A message describing the error: The value thrown is the exception object , which contains details about the error and can be accessed in the catch block

04*Some of Java's Important Exceptions:

ArithmeticException	Division by zero or some other arithmetic problem
ArrayIndexOutOfBoundsException	An array index is less than zero or greater than or equal to the array's length
FileNotFoundException	Reference to a file that cannot be found
NumberFormatException	Use of an illegal number format , as when calling a method
StringIndexOutOfBoundsException	A String index is less than zero or greater than or equal to the String's length

NullPointerException: Reference to an object that has not been instantiated

```
String name = null;
System.out.println(name.length());
```

Output
Exception in thread "main" java.lang.NullPointerException

indexOutOfBoundsException: An array of String index is out of bounds

```
int[] number = {1,2,3} ;
System.out.println(number[5]);
```

Output
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: Index 5 out of bounds of length 3

Handling NullPointerException :

```
public class Main {
  public static void main(String[] args) {
    String name = null;
    if(name != null)
      System.out.println(name.length());
    else
      System.out.println("The object is null , can't call methods on it");
     }
  }
```

Handling IndexOutOfBoundsException :

```
public class Main {
  public static void main(String[] args) {
    int[]  numbers = {1,2,3};
    int index = 5;
    if(index>=0  &&  index<number.length)
      System.out.println(numbers[index]);
    else
      System.out.println("Index out of bounds: " + index);
     }
  }
```

Output
Index out of bounds: 5

InputMismatchException:

```
import java.util.Scanner;
public class Main {
  public static void main(String[] args) {
    Scanner scan = new Scanner(System.in);
    System.out.println("Enter a number: ");
    try {
      int number = scanner.nextInt();
      System.out.println("You entered : "+ number);
    }
    catch(InputMismatchException e) {
      System.out.println("Invalid input ! Please enter an integer");
      }
    }
}
```

IllegalArgumentException: Calling a method with an improper argument

```
public class Main {
  public static void setAge(int age) {
    if(age<0) {
      throw new IllegalArgumentException("Age cannot be negative!");
    }
    System.out.println("Age is: "+ age);
    }
  public static void main(String[] args) {
    setAge(25); //valid
    setAge(-5); // Throw IllegalArgumentException
    }
}
```

Output
Age is: 25 Exception in thread "main" java.lang.IllegalArgumentException: Age cannot be negative!

Checked Exceptions vs Unchecked Exceptions:

Checked Exception:

- A checked exception is an exception that is checked at compile-time
- The compiler forces you to handle it using try-catch or throws
- Usually occurs in operations outside your control (like file I/O , network , database)

Example:

```
import java.io.File;
import java.io.FileNotFoundException;
import java.util.Scanner;
public class CheckedExample {
  public static void main(String[] args) {
    try {
      File file = new File("data.txt");
      Scanner scanner = new Scanner(System.in);
      }
   }
}
//Here ,FileNotFoundException is a checked exception , the compiler won't let you ignore it
```

Unchecked Exception:

- An unchecked exception is an exception that occurs at runtime
- The compiler does not check whether you handle it or not
- Usually occurs due to programming mistakes (like null pointer , divide by zero , array index out of bounds).

Example:

```
public class UncheckedExample {
  public static void main(String[] args) {
    int[] numbers = {1,2,3} ;
    System.out.println(numbers[5]); //ArrayIndexOutOfBoundsException
    }
}
```

05*The Exception Class:

The Exception class in Java is the base class for all checked exceptions , representing errors that can be handled by the program

Example:

```
public class Main {
  public static void divide(int a , int b) {
    try {
      if(b==0) {
        throw new Exception("Division by zero is not allowed!);
      }
      System.out.println("Result: "+(a/b)) ;
    }
    catch(Exception e) {
      System.out.println("Caught Exception: "+ e.getMessage());
    }
}
  public static void main(String[] args) {
    divide(10,2) ; //Valid division
    divide(10,0) ; //Throws Exception
    }
}
```

Declaring Exceptions like:

--> public void myMethod() throws IOException

--> public void myMethods() throw IOException , OtherException

06*Throwing Exceptions:

Throwing exceptions is the process of using the throw keyword to explicity signal an error during program execution

Example:

```
public class Main {
  public static void checkNumber(int number) {
    if(number<0) {
      throw new IllegalArgumentException("Number cannot be negative!);
     }
    System.out.println("Valid number : " + number);
  }
  public static void main(String[] args) {
    checkNumber(5); //valid
    checkNumber(-3) ; //Throws exception
    }
  }
```

07*File Class:

What is File class?

```
import java.io.File;
```

Can be used to:
Check if a file/directory exists
Create new files or directories
Delete files
Get file properties(name, path , size , readability , writability)

Creating a File Object:

```
File f1 = new File("data.text"); //file in project folder
File f2 = new File("C:/Users/Jawad/Documents/file.text") ; //absolute path
File f3 = new File("C:/Users/Jawad/Documents" , "file.txt") ; //parent + child
```

Common Methods with Examples and Output:

Check if file exists:

```
File f1 = new File("data.txt");
if(f1.exists()) {
System.out.println("File exists!");
   }
else {
System.out.println("File does not exists!") ;
  }
```

Example Output (if file exists) :

```
File exists!
```

Example Output (if file doesn't exists):

```
File does not exit!
```

Check if it's a file or directory:

```
File f1 = new File("example.txt") ;
if(f1.isFile()) {
System.out.println("This is a file");
 }
if(f1.isDirectory()) {
System.out.println("This is a directory");
 }
```

Example output (data.text is a file):

```
This is a file
```

Create a new file:

```
import java.io.IOException;
try {
  if(f1.createNewFile()) {
    System.out.println("File created successfully");
  }
   else {
    System.out.println("File already exists");
  }
}
 catch(IOException e) {
  e.printStackTrace();
  }
```

Example Output (if file already exists) :

```
File already exists
```

Example Output (if file does not exists):

```
File created successfully
```

Delete a file:

```
File f1 = new File("example.txt");
if(f1.delete()) {
 System.out.println("File deleted successfully");
}
else {
 System.out.println("File to delete file");
  }
```

Example Output (if deletion successful):

```
File deleted successfully
```

Get file properties:

```
File f1 = new FIle("example.txt");
System.out.println("File name: "+ f1.getName());
System.out.println("Path: " + f1.getPath());
System.out.println("Absolute path: "+ f1.getAbsolutePath());
System.out.println("Readable : "+ f1.canRead());
System.out.println("Writable: " + f1.canWrite());
System.out.println("File size in bytes: " + f1.length());
```

Example Output:

```
File name: example.txt
Path : example.txt
Absolute path : C:\Users\Jawad\project\example.txt
Readable : true
Writable : true
File size in bytes : 0
```

Chapter 11 : Abstract Classes and Interfaces

01*Abstract Classes:

Abstract classes are classes that cannot be instantiated directly and are declared using the abstract Keyword. They are used to provide a common blueprint for subclasses , containing both implemented methods are abstract methods that must be overridden by subclasses . While objects cannot be created directly from an abstract class , objects can be created from subclasses that extends it , inheriting its properties and behavior

Note : the use of keyword abstract in the class header and the methods header

02*Abstract Class GeometricObject:

```
public abstract class GeometricObject {
   private String color;
   private boolean filled;
   public GeometricObject(String c , boolean f) {
     color = c;
     filled = f;
  }
   public String getColor() {
     return color;
}
   public void setColor(String c) {
     color = c;
  }
   public boolean isFilled() {
     return filled;
  }
   public String toString() {
     String s = "The color is " + color ;
       if(filled)
         s+= ".\nThe geometric object is filled" ;
       else
         s+= "\nThe geometric object isn't filled" ;
       return s;
     }
 public abstract double area() ;
 public abstract double perimeter() ;
  }
```

03*Why Abstract Methods?

Abstract methods are used when a method's behavior must be defined by subclasses , ensuring that each subclass provides its specific implementation . They serve as a contract that enforces consistency across all subclasses inheriting form the abstract class

Example:

```
abstract class Shape {
  abstract double calculateArea();
  }
class Circle extends Shape {
  double radius;
  Circle(double radius) {
    this.radius = radius;
```

```
  }
  @Override
  double calculateArea() {
    return Math.PI*radius*radius ;
    }
}
class Rectangle extends Shape {
  double width , height ;
  rectange(double width , double height) {
    this.width = width ;
    this.height = height ;
  }
  @Override
  double calculateArea() {
  return width*height ;
    }
}
```

```
public class Main {
  public static void main(String[] args) {
    Shape c = new Circle(5);
    Shape r = new Rectangle(4,6) ;
    System.out.println("Circle Area: " + c.calculateArea());
    System.out.println("Rectange Area: " + r.calculateArea());
    }
}
```

Output
Circle Area : 78.53981633974483 Rectange Area : 24.0

04*Interesting points on Abstract classes:

.Abstract classes are like regular classes , but you cannot create instances of abstract classes using the new operator . Hence , the following statement is not allowed :

```
GeometricObject g = new GeometricObject("While" , true) ;
```

.An abstract method is defined without implementation . Its implementation is provided by the subclasses . If a subclass of an abstract superclass doesn't implement all the abstract methods , the subclass extended from an abstract class , all the abstract methods must be implemented

.A class that contains abstract methods must be abstract . However , it is possible to define an abstract an abstract class that contains no abstract methods

.An abstract method cannot be contained in a nonabstract(concrete class)

05*Interfaces:

What is an interface and why is an interface useful?

An interface is a class like construct that contains only constants and abstract methods . In many ways , an interface is similar to an abstract class , but the intent of an interface is to specify a behavior for objects . For example , you can specify that the objects are comparable using the appropriate interface

An interface is treated like a special class in Java . Like an abstract class , you cannot create an instance from an interface using a new operator , but in the most cases you can use an interface more or less the same way you use an abstract class .For example , you can use an interface as a data type for a variable

Define an Interface:

```
public interface interfaceName {
/*Describe how to eat*/
public abstract String howToEat() ;
 }
```

Example:

```
public interface Edible {
  public abstract String howToEat() ;
 }
abstract class Animal {
 }
class Tiger extends Animal {
 }
abstract class Fruit implements Edible {
 }
class Chicken extends Animal implements Edible {
   public String howToEat() {
     return "Chicken : Fry it" ;
   }
}
class Apple extends Fruit {
   public String howtoEat() {
     return "Apple : Make apple cider " ;
   }
}
class Orange extends Fruit {
   public String howToEat() {
     return "Orange : Make orange juice " ;
   }
}
public class TestEdible {
   public static void main(String[] args) {
     Edible[] edibles = {
      new Orange() , new Chicken() , new Apple()
        } ;
      for(int I = 0 ; I<edibles.length ; I++)  {
        //if(edibles[I] instanceof Edible)
        System.out.println(edibles[I].howToEat()) ;
       }
   }
```

06*Omititng Modifiers in Interfaces:

public interface T1 {
public static final int k = 1 ;
public abstract void p() ;
}
	public interface T1 {
	int k = 1 ;
	void p() ;
	}

Equivalent	

07*The comparable Interface:

Package java.lang:

```
public interface Comparable<E> {
   public int compareTo(E o);
 }
```

Application:

```
public class Application {
   public static void main(String[] args) {
     Circle c1 = new Circle(1);
     Circle c2 = new Circle(2);
     Circle c3 = new Circle(3);
     int x = c1.compareTo(c2) ;
     System.out.println("Comparing c1 and c2:");
     if(x==0)
       System.out.println("The two circles are equals") ;
     else if(x<0)
       System.out.println("The first circle is smaller");
     else
       System.out.println("The first circle is larger");
     if(c2.compareTo(c3) == 0)
       System.out.println("The two circles are equal");
     else if(c2.compareTo(c3) <0)
       System.out.println("The first circle is smaller");
     else
       System.out.println("The first circle is larger");
   }
}
```

Chapter 12 : Inner and Nested Classes in Java

01*Definition:

A nested class is a class defined inside another class
If the nested class is non-static , it is called and increase encapsulation
Nested classes help logically group classes that are only used in one place and increase encapsulation

02*Types of Nested Classes:

A)Member Inner class:

Declared inside another class but outside methods
Needs an object of the outer class to be created.

Example:

```
class Outer {
 private String msg = "Hello from Outer" ;
  class Inner {
    void show() {
        System.out.println(msg) ; // can access private members
      }
    }
}
public class TestInner {
    public static void main(String[] args) {
    Outer outer = new Outer();
    Outer.Inner inner = outer.new Inner();
    inner.show();
    }
}
```

Output
Hello from Outer

b)Static Nested Class:

Declared static inside another class.
Can be created without an instance of the outer class
Can only access static members of the outer class

Example:

```
class Outer {
 static String msg = "Hello from Static Nested Class" ;
 static class Inner {
    void show() {
       System.out.println(msg);
        }
   }
}
public class TestStaticNested {
   public static void main(String[] args) {
     Outer.Inner inner = new Outer.Inner() ;
     inner.show();
      }
}
```

Output
Hello from Static Nested Class

c)Local Inner Class:

Declared inside a method
Its scope is limited to that method

Example:

```
class Outer {
  void display() {
     class LocalInner {
         void msg() {
            System.out.println("Hello from Local Inner Class");
           }
        }
    LocalInner local = new LocalInner();
    local.msg();
  }
}
public class TestLocalInner {
   public static void main(String[] args) {
     Outer outer = new Outer() ;
     outer.display();
   }
}
```

Output
Hello from Local Inner Class

d)Anonymous Inner Class:

A class without a name

Used for short implementations , often with interfaces or abstract classes

Example:

```
abstract class Animal {
   abstract void sound() ;
   }
public class TestAnonymousInner {
   public static void main(String[] args) {
     Animal a = new Animal() {
       void sound() {
        System.out.println("Dog barks");
       }
     };
     a.sound();
   }
}
```

Output
Dog barks

Chapter 13: Enum

01*Definition:

An Enum (short for enumeration) is a special Java type used to define a fixed set of constants

Useful when you have a limited list of possible values (like days of the week , sizes , directions).

02*Basic Example:

```
enum Size {
   SMALL , MEDIUM , LARGE
  }
public class TestEnum {
  public static void main(String[] args) {
    Size s = Size.MEDIUM ;
    System.out.println("Selected size:" + s);
   }
}
```

Output
Selected size : MEDIUM

03*Enum with Fields and Methods:

```
enum Size {
  SMALL(1) , MEDIUM(2) , LARGE(3) ;
  private int value;
  Size(int value)  {
    this.value = value;
   }
  int getValue() {
   return value;
  }
public class TestEnumAdvanced {
   public static void main(String[] args) {
     Size s = Size.LARGE ;
     System.out.println("Size: "+s+", Value : "+s.getValue());
     }
  }
```

Output
Size: LARGE , Value: 3

Translated to a Class :

```
class Size {
  public static final Size SMALL = new Size(1);
  public static final Size MEDIUM = new Size(2);
  public static final Size LARGE = new Size(3);
  private int value;
  private Size(int value) {
      this.value = value;
  }
    public int getValue() {
      return value;
    }
}
public class TestSizeClass {
    public static void main(String[] args) {
      Size s = Size.LARGE;
      System.out.println("Size: "+ s + " , Value: "+ s.getValue());
    }
}
```

Output
Size : Size@<hashcode>, Value: 3